Федеральное государственное автономное образовательное учреждение
высшего образования

**«Национальный исследовательский университет
“Высшая школа экономики”»**

Факультет компьютерных наук
Департамент программной инженерии

Отчёт по индивидуальному заданию № 1

по дисциплине «Архитектура компьютера и операционные системы»

Вариант 14. «Задача об умных цветах»

Выполнила: Гончарова Екатерина

Группа: БПИ-253

Москва, 2026

Содержание

1 Постановка задачи **2**

2 Входные данные **2**

3 Модель и сценарий работы **3**

4 Представление сущностей **3**

5 Структура программы **3**

6 Пользовательский интерфейс **4**

7 Варианты запуска **4**

8 Инварианты модели **5**

9 Журналирование и системный ввод-вывод **5**

10 Тестирование **5**

11 Дополнительные возможности **6**

12 Заключение **6**

1 Постановка задачи

Требуется разработать консольное приложение на языке C для ОС Linux, которое с помощью последовательной событийной имитации моделирует взаимодействие умных цветов и садовников.

На клумбе находится заданное количество цветов. Каждый цветок самостоятельно изменяет своё состояние и при недостатке влаги формирует заявку на полив. Садовники принимают активные заявки и обслуживают их. Если несколько садовников могут откликнуться на одну заявку, заявка должна быть закреплена только за одним садовником. После полива действует защитный интервал, в течение которого повторный полив считается переливом и может привести к гибели цветка.

В модели используются следующие состояния цветка:

- нормальное состояние;
- увядание;
- полив;
- восстановление;
- засыхание;
- гибель от перелива.

Программа должна отображать изменение состояния цветов, создание и снятие заявок, принятие заявки садовником, начало и окончание полива, восстановление, засыхание и перелив. Моделирование завершается после истечения заданного периода, после прекращения активности всех цветов или по запросу пользователя.

2 Входные данные

Набор входных параметров программы соответствует перечню из условия варианта:

- количество цветов;
- индивидуальные скорости высыхания;
- порог формирования заявки на полив;
- порог засыхания;
- длительность полива;
- длительность восстановления;
- защитный интервал от повторного полива;
- количество садовников;
- скорость работы садовников;
- правило выбора заявок.

Дополнительные пользовательские параметры в программу не вводятся. Начальная влажность и период наблюдения являются внутренними константами модели. После успешного завершения полива влажность цветка восстанавливается до максимального значения.

Параметры могут вводиться интерактивно или считываться из файла. Для тестовых запусков используются файлы формата `.cfg` с парами `key=value`. Например:

```
flower_count=6
drying_rates=4,7,5,9,6,8
request_threshold=47
dry_threshold=14
watering_duration=2
```

```
recovery_duration=2
protection_interval=5
gardener_count=2
gardener_speed=2
request_policy=first
```

Для правила выбора заявки поддерживаются два значения: `first` — первая доступная заявка по номеру цветка и `driest` — заявка активного цветка с наименьшей текущей влажностью.

3 Модель и сценарий работы

Один проход главного цикла программы соответствует одному условному такту. Внутри такта действия выполняются последовательно, но состояние каждой сущности хранится отдельно. Это позволяет имитировать независимое поведение цветов и садовников без использования потоков.

На каждом такте выполняются следующие действия:

1. для каждого активного цветка уменьшается оставшийся защитный интервал;
2. цветы, которые не находятся в состоянии полива или восстановления, теряют влагу согласно своей индивидуальной скорости высыхания;
3. при достижении порога заявки цветок переходит в состояние увядания и создаёт заявку, если активной заявки ещё нет;
4. при достижении порога засыхания цветок прекращает активность;
5. свободные садовники выбирают ещё не назначенные заявки;
6. скорость работы садовника определяет, как быстро он завершает подход к выбранному цветку;
7. после подхода начинается полив;
8. по завершении полива влажность восстанавливается до максимума, заявка закрывается, включается защитный интервал и начинается восстановление;
9. после обработки всех участников проверяется условие завершения.

Правило выбора заявки задаётся входным параметром. В режиме `first` садовник выбирает первую доступную заявку по номеру цветка, а в режиме `driest` — заявку цветка с минимальной текущей влажностью.

4 Представление сущностей

Для цветка используется структура `Flower`. Она хранит идентификатор, влажность, индивидуальную скорость высыхания, текущее состояние, наличие активной заявки, назначенного садовника и таймеры полива, восстановления и защитного интервала.

Для садовника используется структура `Gardener`. Она хранит идентификатор, признак занятости, индекс обслуживаемого цветка, оставшееся время текущего действия и скорость работы.

Общие параметры одного запуска объединены в структуру `Settings`. Это позволяет передавать настройки между функциями одной переменной и не использовать глобальные параметры модели.

5 Структура программы

Проект разбит на небольшие модули по ответственности:

Файл	Назначение
src/main.c	Минимальная точка входа программы.
src/app.c	Запуск приложения, выбор способа ввода, проверка параметров, настройка журнала и обработчика Ctrl+C.
src/input.c	Интерактивный ввод и вывод параметров.
src/config_parser.c	Чтение и разбор .cfg-файлов.
src/input_validation.c	Проверка корректности входных параметров.
include/constants.h	Именованные константы модели: границы влажности, максимальные размеры, специальные значения.
src/model.c	Инициализация цветов и садовников, базовые операции модели.
src/flower_logic.c	Изменение влажности и состояний цветка, создание и снятие заявок.
src/gardener_logic.c	Выбор заявок, подход к цветку, полив и освобождение садовника.
src/simulation.c	Главный цикл последовательной имитации.
src/statistics.c	Формирование итоговой статистики.
src/logger.c	Вывод событий в консоль и сохранение журнала в файл.
include/	Заголовочные файлы модулей.
tests/	Подготовленные тестовые конфигурации.

Такое разбиение позволяет отделить предметную логику цветов и садовников от ввода, журналирования и управляющего цикла.

6 Пользовательский интерфейс

Программа является консольной. Без аргументов командной строки параметры последовательно запрашиваются у пользователя. После проверки введённых значений программа печатает параметры запуска и начинает моделирование.

Во время работы выводятся сообщения вида:

```
[07] Цветок 2: NORMAL -> WILTING
[07] Цветок 2: отправлена заявка на полив
[07] Садовник 1: принял заявку цветка 2
[08] Садовник 1: начал полив цветка 2
[10] Садовник 1: закончил полив цветка 2, влажность 24 -> 79
```

Тот же журнал сохраняется в файл simulation.log. Благодаря этому результат можно просмотреть после завершения программы.

7 Варианты запуска

Сборка проекта выполняется командой:

```
make
```

Интерактивный запуск:

```
./smart_flowers
```

Запуск с заранее подготовленным набором параметров:

```
./smart_flowers tests/test1.cfg
```

Также в `Makefile` предусмотрены готовые цели:

```
make test1
make test2
make test3
```

Нормальное завершение происходит после истечения внутреннего периода наблюдения либо если все цветы прекратили активность. Досрочное завершение выполняется по `Ctrl+C`: обработчик сигнала устанавливает флаг остановки, после чего главный цикл корректно завершается и печатает итоговую статистику.

8 Инварианты модели

В реализации контролируются основные ограничения задачи:

- активная заявка закрепляется не более чем за одним садовником;
- садовник одновременно работает не более чем с одним цветком;
- цветок не создаёт новую заявку, пока предыдущая заявка активна;
- завершённая заявка снимается и не может быть обслужена повторно;
- засохший или погибший от перелива цветок больше не участвует в моделировании;
- параметры запуска проверяются до начала моделирования.

9 Журналирование и системный ввод-вывод

Для журнала используется файловый дескриптор. Файл открывается системной функцией `open`, события записываются функцией `write`, а после завершения дескриптор закрывается функцией `close`. Один и тот же текст события отправляется в стандартный вывод и в файл журнала.

Это позволяет сохранить наглядный консольный интерфейс и одновременно использовать ввод-вывод, основанный на файловых дескрипторах.

10 Тестирование

В проект включены три тестовых набора.

Тест 1

Шесть цветов и два садовника с различными скоростями высыхания. Набор проверяет обычный режим работы: постепенное появление заявок, их распределение между садовниками, полив и последующее восстановление цветов.

Тест 2

Семь цветов и три садовника. Повышенные скорости высыхания и более высокий порог формирования заявки приводят к появлению нескольких заявок за небольшой промежуток времени. Тест проверяет обслуживание нескольких активных заявок.

Тест 3

Пять цветов и один садовник. Невысокая скорость работы единственного садовника создаёт очередь из заявок и позволяет проверить ожидание обслуживания, засыхание отдельных цветов и корректное завершение моделирования.

После каждого запуска программа выводит число живых, засохших и погибших от перелива цветов, общее количество поливов, а также итоговое состояние, влажность и число поливов каждого цветка.

11 Дополнительные возможности

Помимо основной логики задачи в проекте реализованы:

- комментарии и Doxygen-документация для структур и функций;
- генерация HTML-документации командой `make docs`;
- запись журнала выполнения в `simulation.log`;
- корректное завершение по `Ctrl+C`;
- подготовленные тестовые наборы и команды их запуска;
- чтение тестовых параметров из конфигурационных файлов;
- проверка корректности входных параметров;
- итоговая статистика по состоянию цветов.

12 Заключение

В результате разработана последовательная имитационная программа на языке C, моделирующая взаимодействие умных цветов и садовников. Состояние участников хранится независимо, а их действия выполняются по тактам в определённом порядке. Программа отражает основные события предметной области, контролирует ключевые ограничения, поддерживает интерактивный и файловый ввод, сохраняет журнал работы и корректно завершает выполнение как штатно, так и по пользовательскому прерыванию.